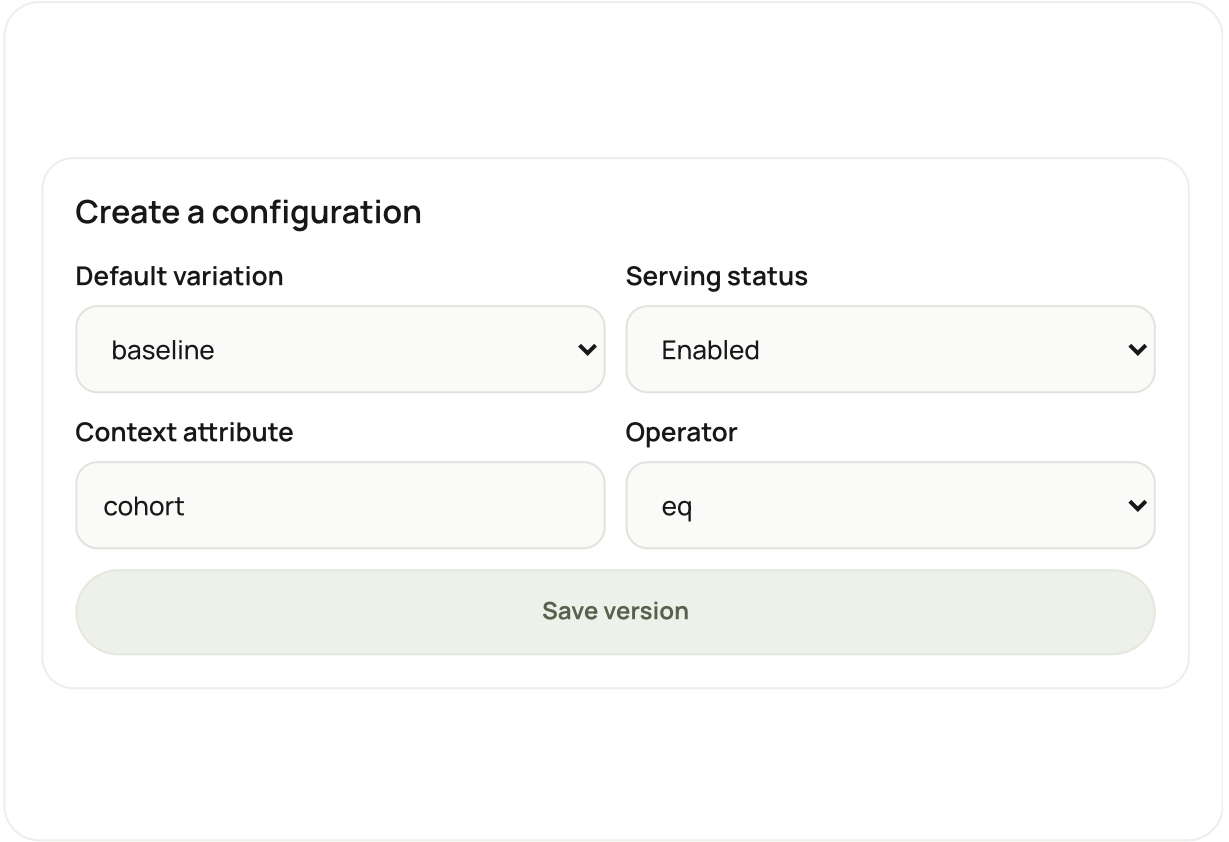
Your workspace, explained.

A practical guide to configuring, testing, and releasing your AI features.

Feature flags AI runtime Workspace

Open Control Studio ↗



Vector controls · Actual product styling

Find your next step.

01 Start, navigation and connection

02 Boolean and multivariate feature flags

03 AI configurations and model variations

04 Reusable models, prompt snippets and resource history

05 Targeting rules and audience segments

06 Judges, datasets and calibration

07 Playground preview, streaming and images

08 Tools, agents and multi-agent graphs

09 Offline jobs, benchmarks and recovery of jobs

10 Online scoring, outcomes and execution SDK

11 A/B experiments, holdouts, bandits and sample planning

12 Progressive rollouts, guards and automatic rollback

13 Adaptive triggers and one-step fallback retries

14 History, stop, approvals, release plans and schedules

15 Monitoring, business metrics, audit and export

16 Traces and OpenTelemetry Collector

17 Notifications, Slack delivery and retention

18 Provider credentials and live validation

19 API credentials, accounts and permissions

20 MFA, recovery codes, password and SSO

21 SCIM Users and Groups

22 Workspace execution policy

23 SDKs, frameworks, watch and signed cache

24 Self-hosted deployment, backup and recovery

25 API access, troubleshooting and verification

26 Record a safe walkthrough

New here? Read the credential setup first. The index links jump directly to each feature.

Three keys. Different jobs.

You do not need every credential on day one. Start with the workspace token; add a provider key when you want a real AI answer.

Workspace token

Opens the studio.
Your local owner token connects this app to your workspace. In this installation, find it in `%LOCALAPPDATA%\ControlStudio\workspace-token.txt`. Paste it into **Workspace token** and click **Connect workspace**.

Provider API key

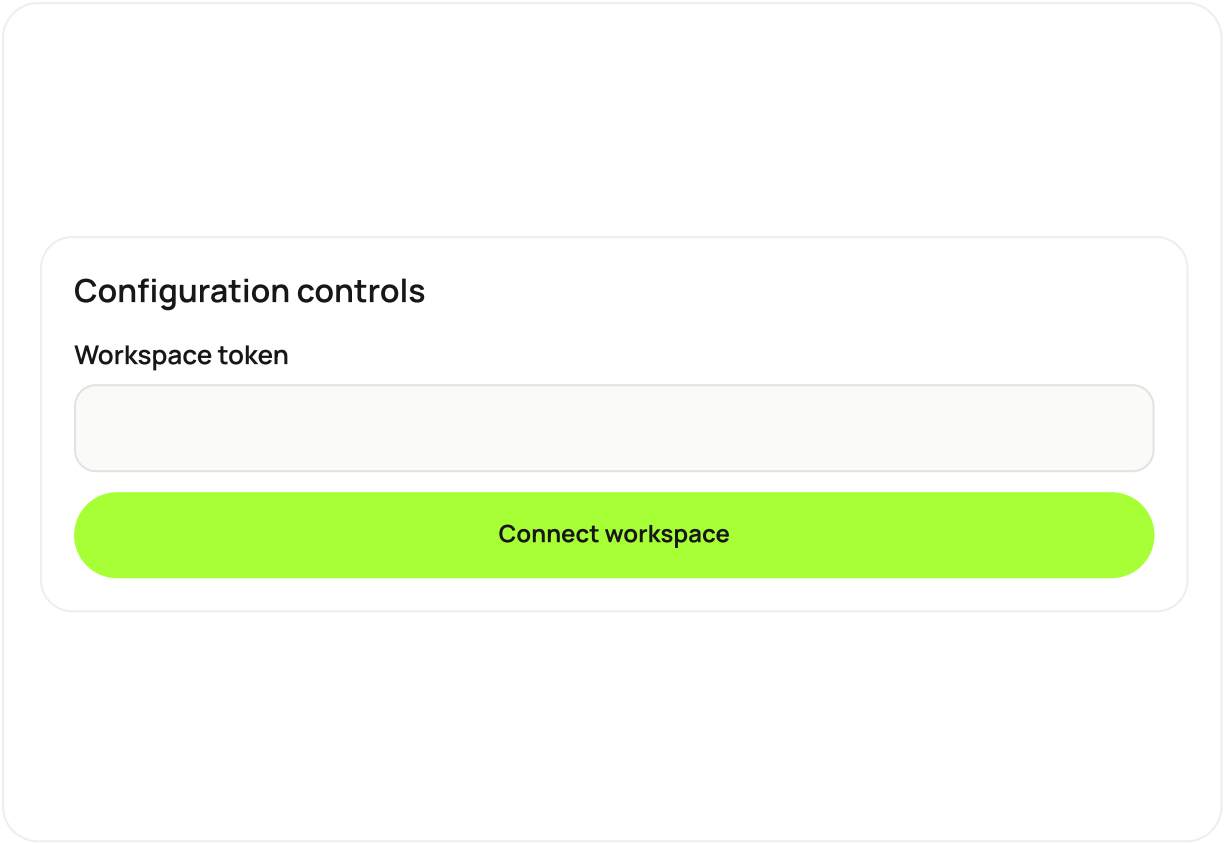
Connects the model.
Get a key from your OpenAI, Anthropic or Gemini provider account. Save it under **Workspace → Access & settings → Provider credentials**. A chat subscription is separate from an API key.

Scoped API token

Connects your own app.
Create an expiring token under **API credentials**. Give it only the environments, resources and role your integration needs. Copy its one-time value into your app's protected server environment.

Start small. You can configure flags, edit prompts and resolve a selection without paying for model generation. Real generation and LLM judges can incur provider charges.

Connect once. Choose your space.



Connect with your protected workspace token. The printed example contains no usable credential.

For a fresh server started without `FLAG_ADMIN_TOKEN`, the terminal prints a new owner token. Restarting without a persistent token changes it.

1. Open the running project at http://127.0.0.1:4315/. Choose Workspace in the shared header.
2. Paste your workspace token into the connection field and choose Connect workspace. Confirm the connected message.
3. Choose Development for your first test. Use Feature flags for ordinary flags, AI runtime for prompts/models, and Workspace for evaluation, releases and access.

Open workspace access ↗

Get your first real answer.

1. In Workspace → Access & settings → Provider credentials, choose a provider and save its API key locally.
2. In AI runtime → Configure, create an AI configuration. Choose a supported model ID, add a system prompt, set a variation weight and save the version.
3. Open Test. Resolve first to check the selected configuration. Then enter an input and explicitly enable provider calls before Generate. Inspect the returned output and provider error details.

Example starting model: `gpt-4.1-mini` for your OpenAI smoke test, if your account supports it. Claude uses an Anthropic key; Gemini uses a Gemini key.

Resolve selects. Generate answers.

Resolve checks which variation a user receives. It does not ask a model to write a completion.

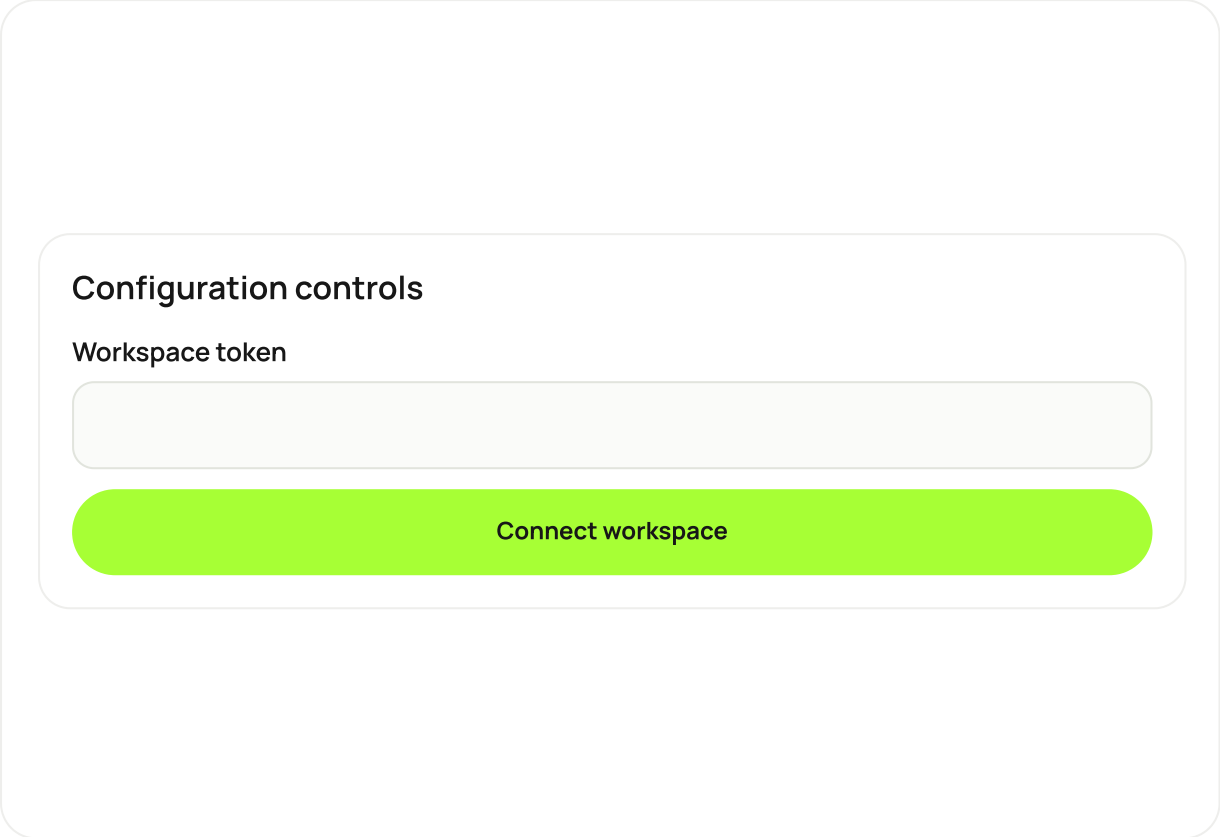
1. Provider key saved
2. Configuration version saved
3. Provider calls enabled

Open AI runtime ↗

If no completion is generated, check these three items before changing the prompt.

Start, navigation and connection

Open http://127.0.0.1:4315/ with the local server running. The shared header contains Feature flags, AI runtime and Workspace. The application currently uses the requested 67% global scale.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

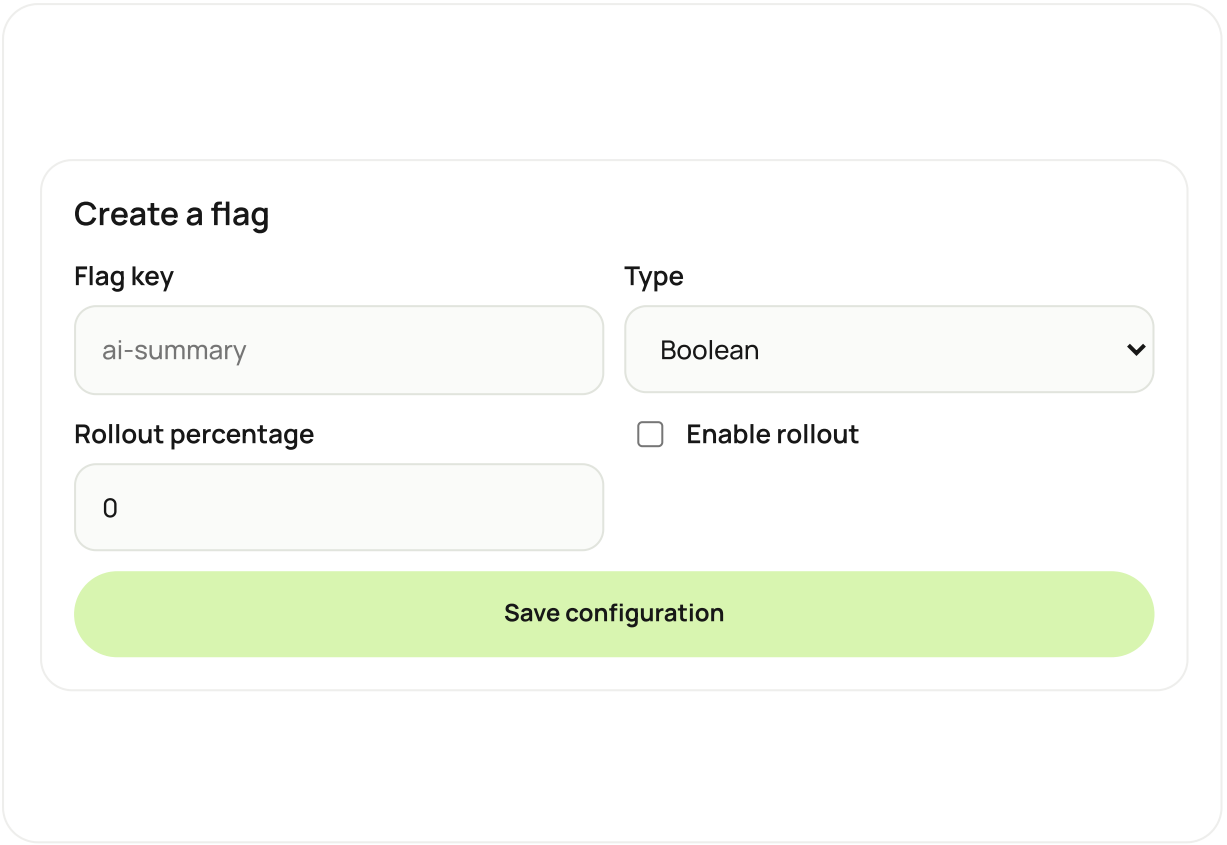
- **What you should see**
  Connected status is visible and the intended environment is selected.

1. Choose Workspace and connect with the owner or managed token supplied by your administrator. Manual tokens remain in page memory; never put them into recordings or this PDF.
2. Alternatively expand Sign in with an account and enter the workspace, actor, password and required authenticator/recovery code. The one-hour account session carries across pages.
3. Select Development, Staging or Production before editing resources. Sign out ends the interactive session. Disabled controls can indicate disconnection, insufficient role or production restrictions.

Open this part of the app ↗

Boolean and multivariate feature flags

Location: Feature flags. Flags control application behavior; evaluating a flag does not call a model.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

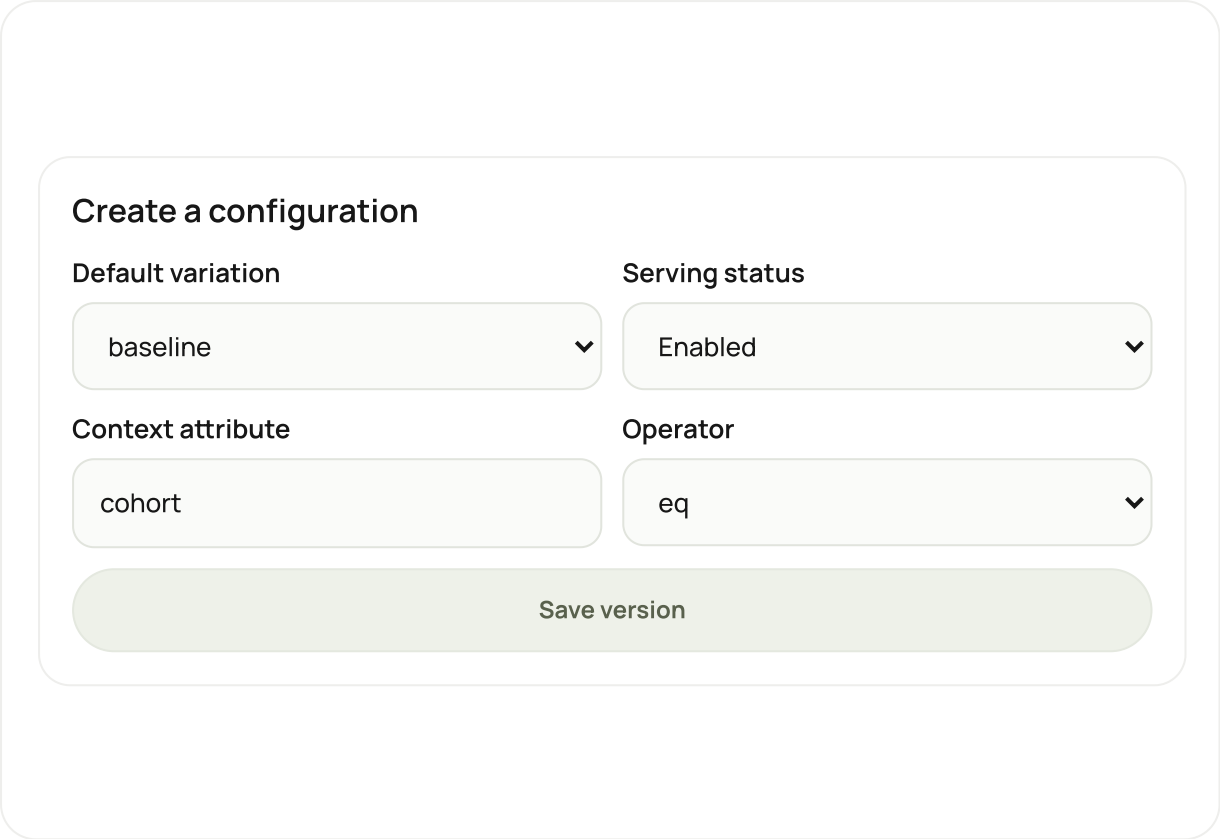
- **What you should see**
  A saved version appears in history; a test decision explains which value was served.

1. Choose New and enter a lowercase key of 2-64 characters starting with a letter. Set Boolean or Multivariate, rollout percentage, enabled state and optional target cohorts.
2. For Boolean, choose enabled and default values. For Multivariate, provide a default variant and JSON values with weights totaling 100. Save configuration to create an immutable version.
3. Search and select flags, evaluate with a stable subject ID and matching cohorts, then inspect the decision reason, history and audit. Stop rollout returns traffic to the default. Restore creates a new version.

Open this part of the app ↗

AI configurations and model variations

Location: AI runtime › Configure. Use structured fields or Advanced configuration for full JSON.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

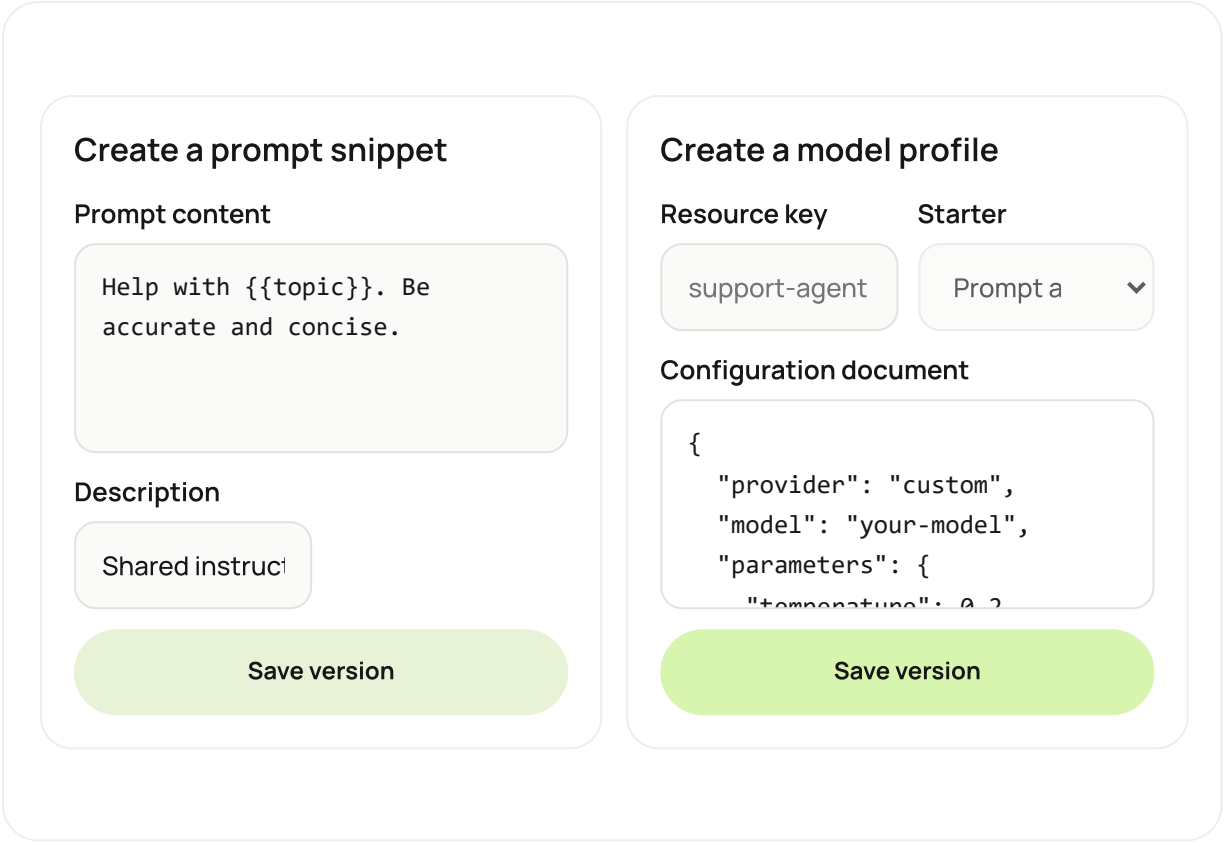
- **What you should see**
  The configuration has a saved version with valid variation weights and model settings.

1. Select AI configurations, New and a starter. Completion mode uses messages; agent mode uses instructions. Enter a resource key, serving state and default variation.
2. Set variation IDs, weights totaling 100, provider, actual account-supported model ID and parameters. Configure pricing per million input/output tokens only from verified rates.
3. Attach judges, targeting or a supported release policy, then Save version. Refresh after version conflicts. Existing saved versions retain pinned model/tool/snippet/segment definitions.

Open this part of the app ↗

Reusable models, prompt snippets and resource history

Location: AI runtime › Configure › Resource type.



1. Create Model profiles with provider, model and parameters. Reference them from variations instead of duplicating settings.
2. Create Prompt snippets with content and description. Link snippet keys and optional versions to messages or agent instructions. Use {{variable}} placeholders and supply matching runtime variables.
3. Use resource history to inspect or restore. Editing a library does not silently update a saved configuration: save a new config version to adopt it. Promotion remaps dependencies into destination version history.

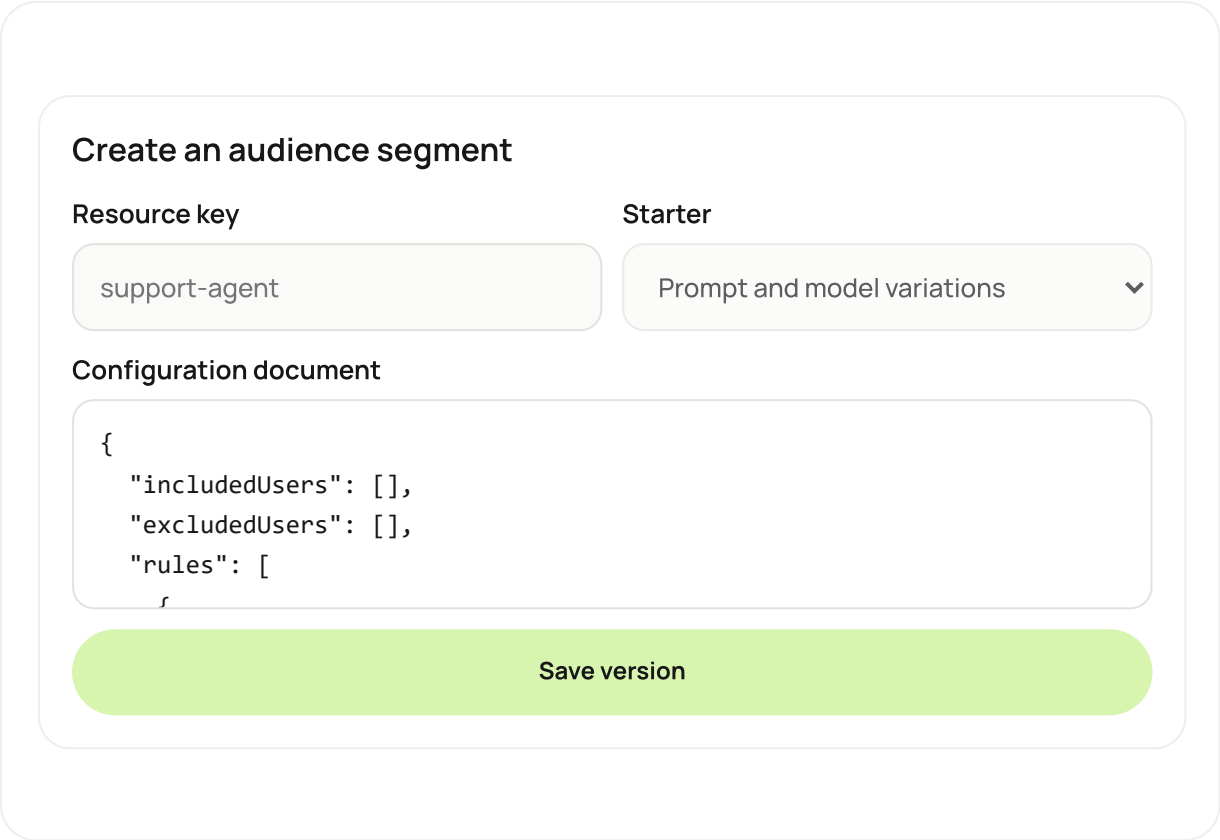
Open this part of the app ↗

Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

- **What you should see**
  Reusable library versions are saved and explicitly referenced by a configuration.

Targeting rules and audience segments

Location: Audience segments and configuration Audience targeting.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

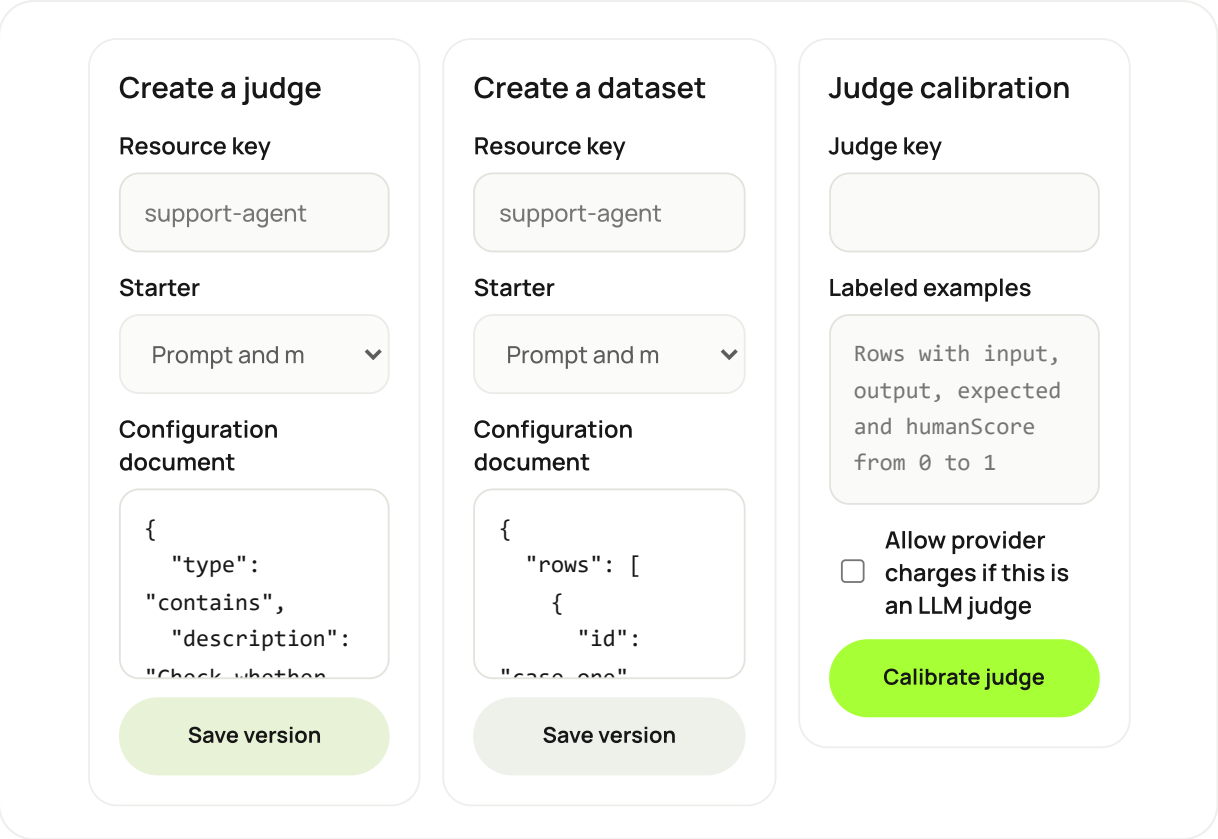
- **What you should see**
  Matching and nonmatching subjects resolve to the expected variations.

1. Create inclusions, exclusions and attribute rules. Segment exclusions take precedence; segment attribute rules are OR-matched.
2. Add configuration targeting rules in priority order using eq, in, gte, lte or contains. First match serves the chosen variation; unmatched subjects use weighted allocation.
3. Preview matching and nonmatching subjects with stable IDs and attributes. Save a new configuration to adopt a changed segment. Targeted/preview traffic is excluded from fixed-randomization winner analysis.

Open this part of the app ↗

Judges, datasets and calibration

Location: Evaluation judges/datasets; Workspace › Evaluations › Judge calibration.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

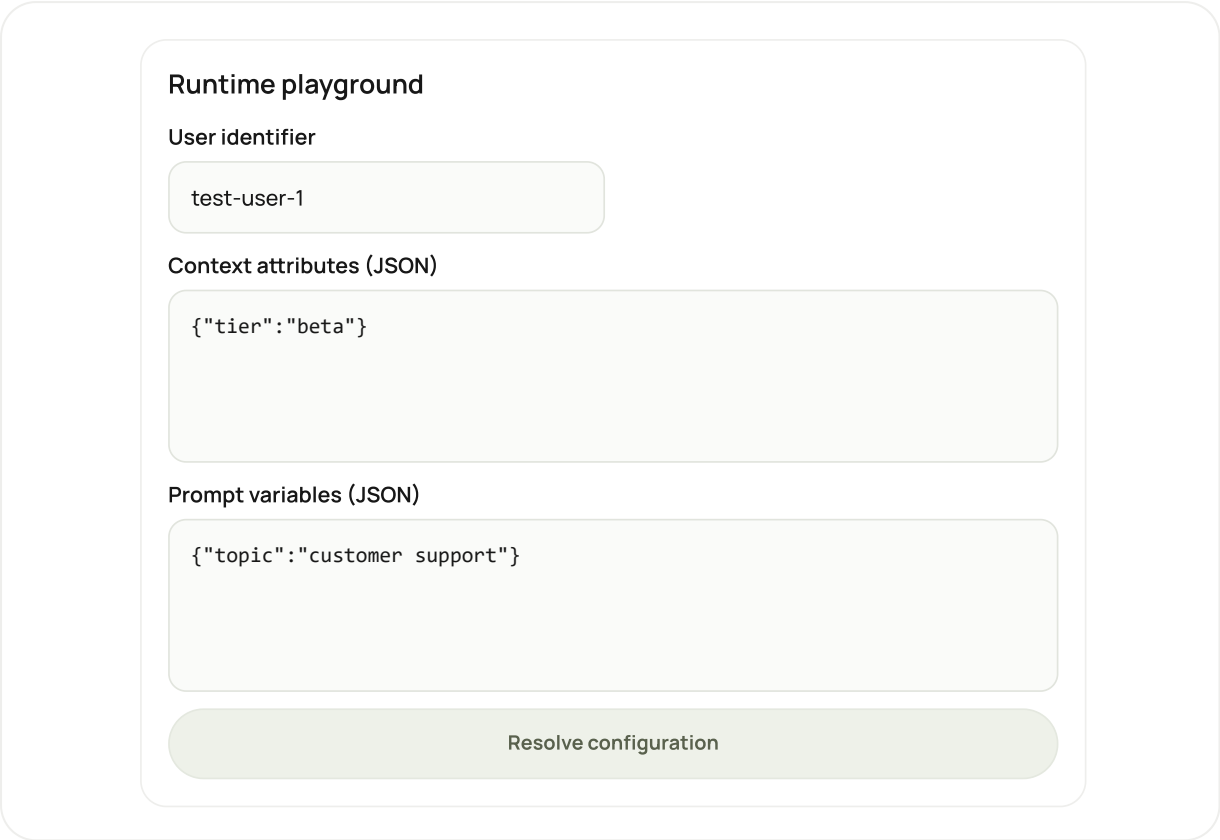
- **What you should see**
  The judge and dataset are saved; calibration evidence is available when run.

1. Create contains or exact judges for expected text, or an LLM judge with provider/model and a rubric requesting JSON score (0-1) and reason. Higher scores must mean better quality. Attach up to five judges.
2. Create dataset rows with unique IDs, input, expected output, variables and optional tags/image attachments. Limits: 1000 rows within 4 MB; durable jobs up to 1000 row/variation pairs.
3. Calibrate a pinned judge using 3-100 human-labeled examples (input, output, expected, humanScore). Explicitly authorize LLM charges. Inspect mean absolute error and agreement within 0.2; recalibrate after rubric changes.

Open this part of the app ↗

Playground preview, streaming and images

Location: AI runtime › Test.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

- **What you should see**

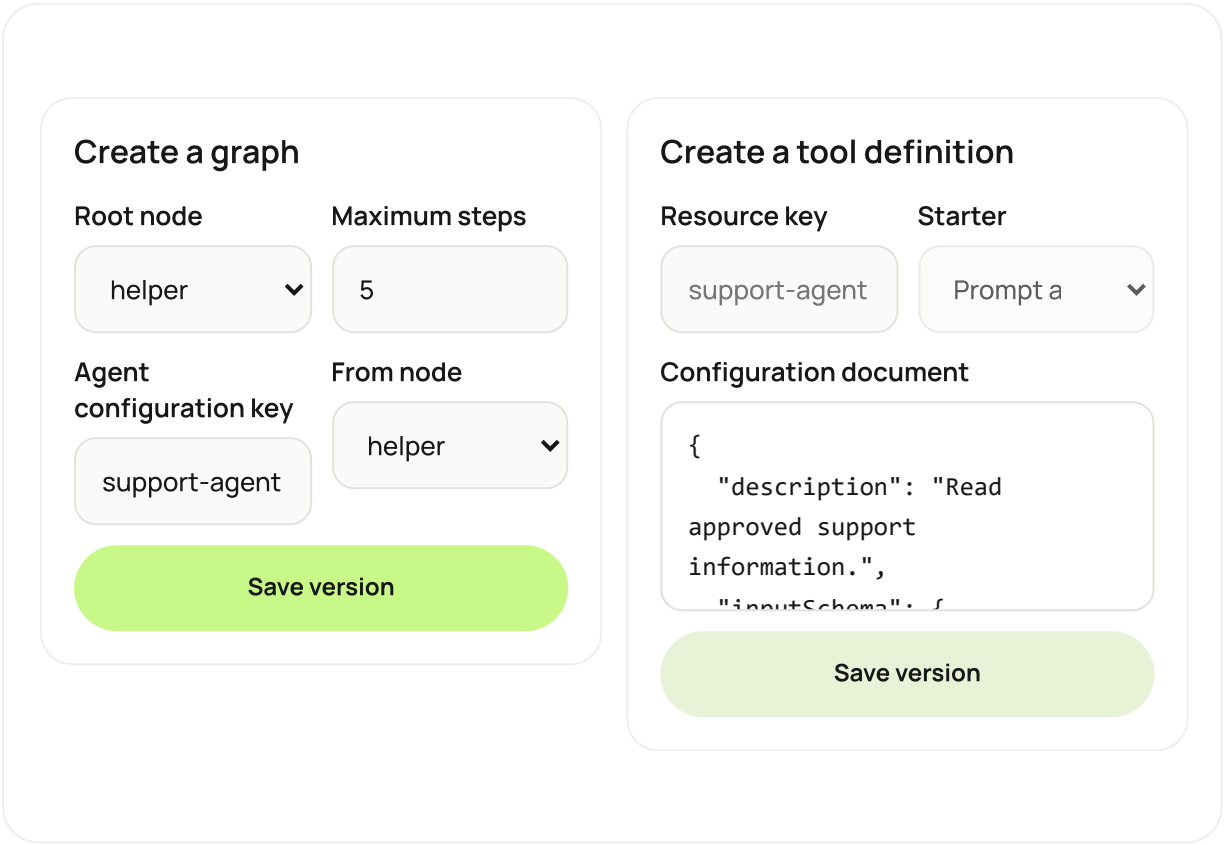
  Resolve returns the selected variation; authorized generation returns output or a specific provider error.

1. Resolve or preview with context and variables to inspect selection without calling a provider. Choose a saved config/variation for actual generation.
2. Configure credentials first and explicitly allow provider calls. Enter input/expected answer. Compatible OpenAI, Anthropic, Gemini and Bedrock models support native streaming, images and tool loops.
3. Images are bounded PNG/JPEG/WebP: up to three, 500 KB each. Inspect final output, usage and judges; a truncated stream is not successful completion. Cost is unknown when token usage or configured pricing is missing.

Open this part of the app ↗

Tools, agents and multi-agent graphs

Location: Tool definitions, agent configurations and Agent graphs.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

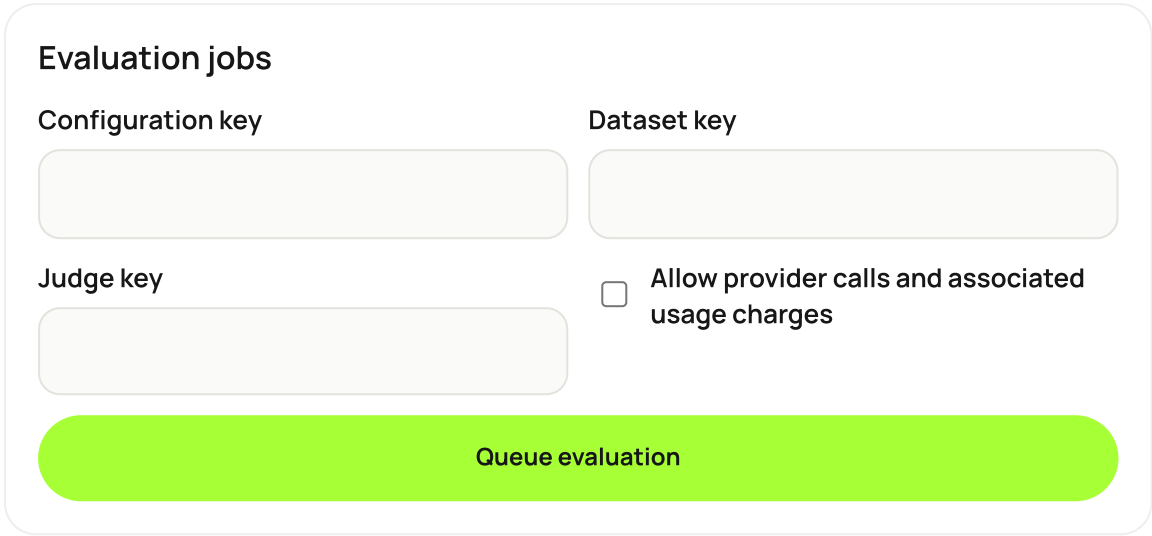
- **What you should see**
  Registered tools or graph nodes execute with bounded permissions and recorded evidence.

1. Save tool descriptions, schemas and approval requirements, then register real handlers in createApp toolHandlers or your executor. The bundled read-only calculator supports sum, mean, min and max.
2. Opt into tool execution and name approval-required tools. Supported schemas include typed properties, required, enum, numeric bounds and string/array lengths; unsupported constraints fail. Native execution is capped at six turns and ten tool calls.
3. Create graph nodes referencing agent-mode configs, choose root, valid edges and max steps. Execute through RuntimeClient.runGraph with application-owned edge selection and bounded handoff state/input. Inspect graph metrics; cycles are bounded, orphan nodes rejected.

Open this part of the app ↗

Offline jobs, benchmarks and recovery of jobs

Location: Workspace › Evaluations › Evaluation jobs.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

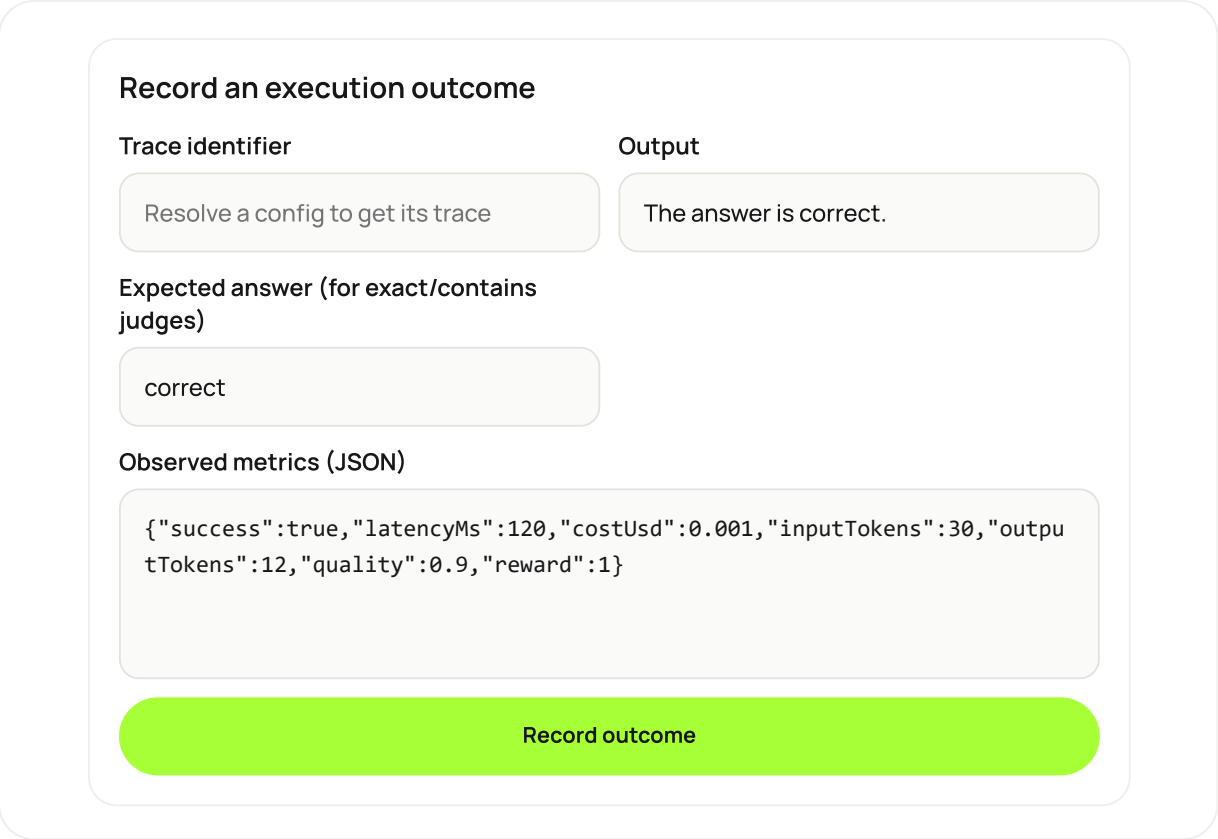
- **What you should see**
  The job has a visible status and pinned inputs, with results or an actionable error.

1. Select config, dataset and judge, then explicitly allow provider calls. For tool-enabled evaluations register handlers and list approvals before Queue evaluation.
2. Refresh activity and inspect status, pinned versions, results and tagged slices. Cancel stops further work after the in-flight request; Resume retains completed checkpoints.
3. Unfinished jobs recover after restart. For app-generated outputs use benchmark submission; for generated benchmarks use the SDK/provider benchmark path. A crash before checkpointing can repeat a billable request.

Open this part of the app ↗

Online scoring, outcomes and execution SDK

Location: integrating application and AI runtime monitoring.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

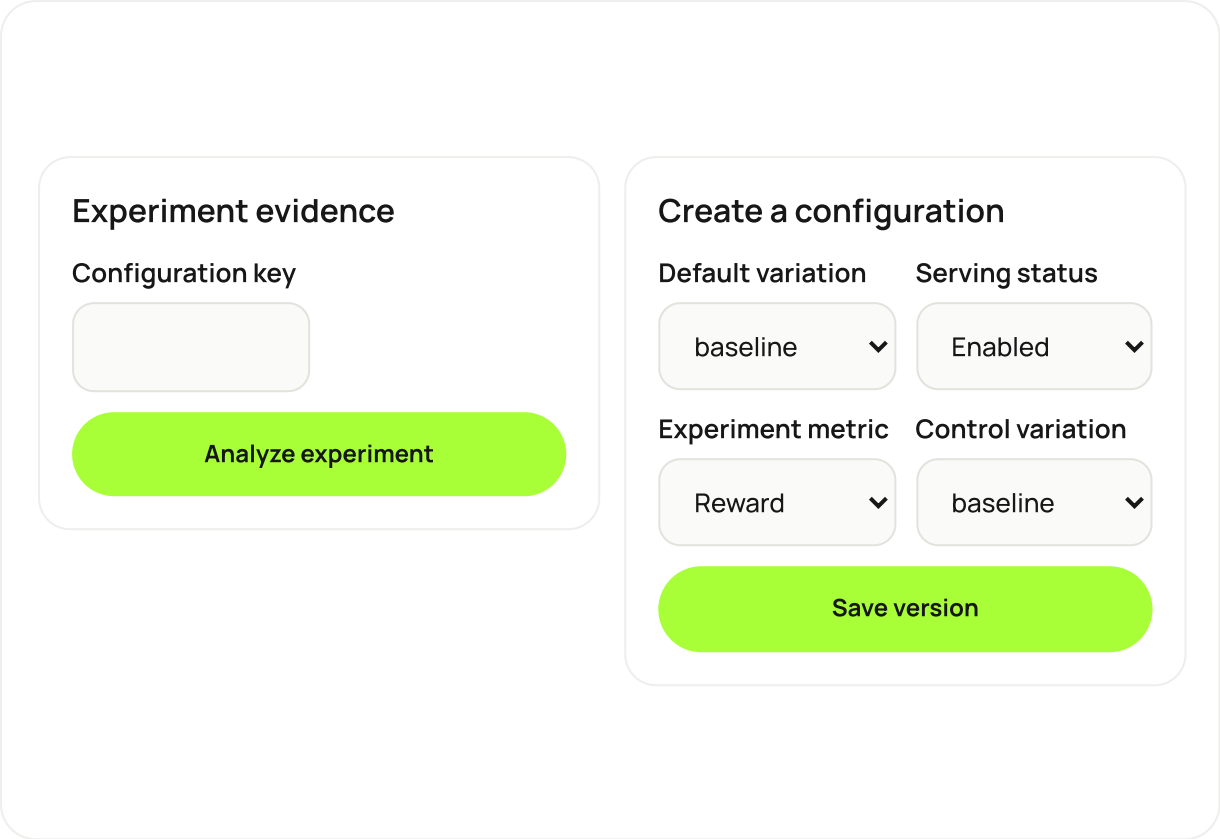
- **What you should see**
  An execution result and its linked outcome appear in monitoring.

1. Construct RuntimeClient with base URL, environment, scoped token and providers/executor. Call execute with context.userId, attributes, variables and input; include expected for deterministic judges.
2. Pass attachments, onEvent, toolHandlers and approvedTools as needed. Honour cancellation in custom executors. Inspect output, attempts, trace IDs, versions, metrics and judgement evidence.
3. Submit outcomes against existing execution IDs. Check telemetryError on failure; retry is not issued without recording the first outcome. Generation cost excludes separately executed LLM judge charges.

Open this part of the app ↗

A/B experiments, holdouts, bandits and sample planning

Location: AI configuration experiment controls; Workspace › Experiment evidence.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

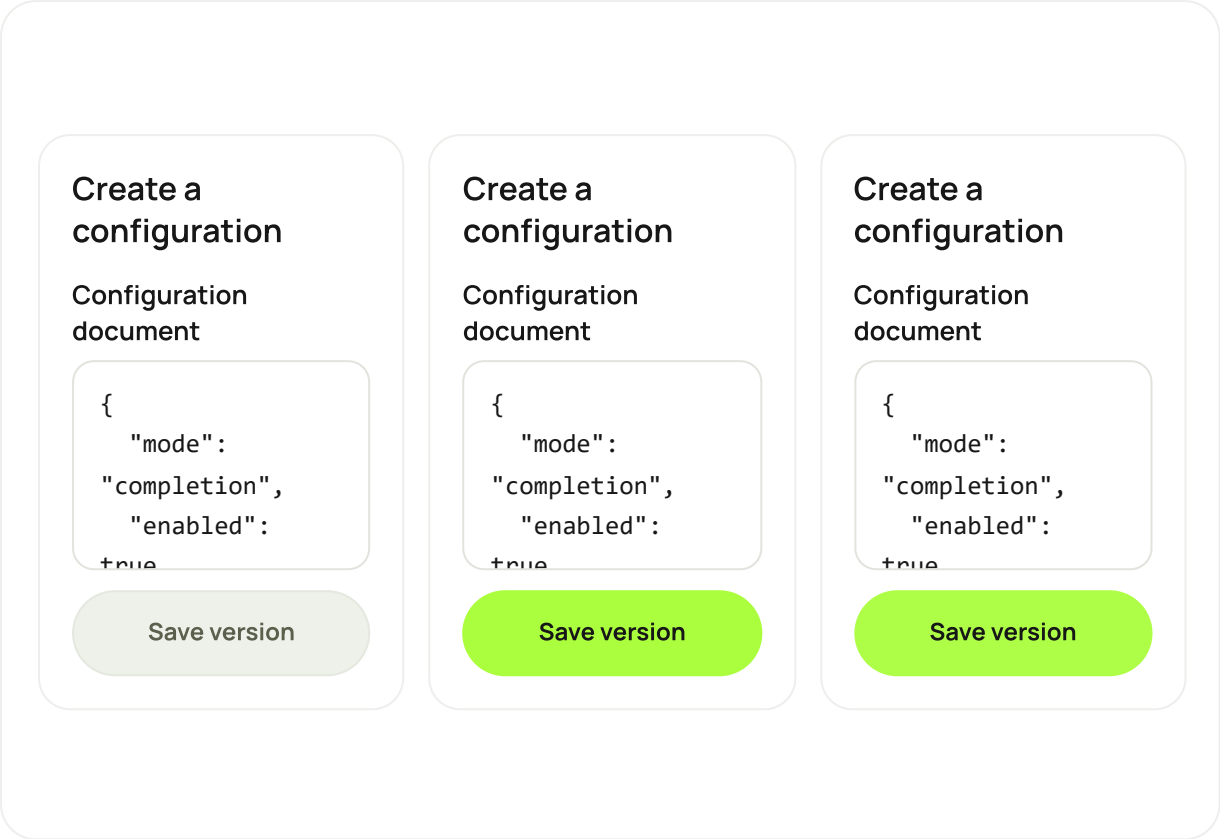
- **What you should see**
  The experiment analysis shows sample sizes, confidence and whether evidence is conclusive.

1. For fixed A/B, set control, weights, metric (quality/reward/satisfaction/error rate), independent minimum subjects, confidence and optional stable holdout. Report valid outcomes with stable subject IDs.
2. Inspect per-arm samples, intervals and allocation checks. Use sample planning for conservative precision estimates. Automatic promotion is opt-in and requires a winner that beats every active arm; ties do not arbitrarily pick a winner.
3. Bandit uses epsilon-greedy learning from 0-1 reward with pinned assignments; it does not establish a fixed-randomization winner. Experiments cannot coexist with guards, triggers or progressive rollout on the same configuration.

Open this part of the app ↗

Progressive rollouts, guards and automatic rollback

Location: configuration release controls / Advanced configuration.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

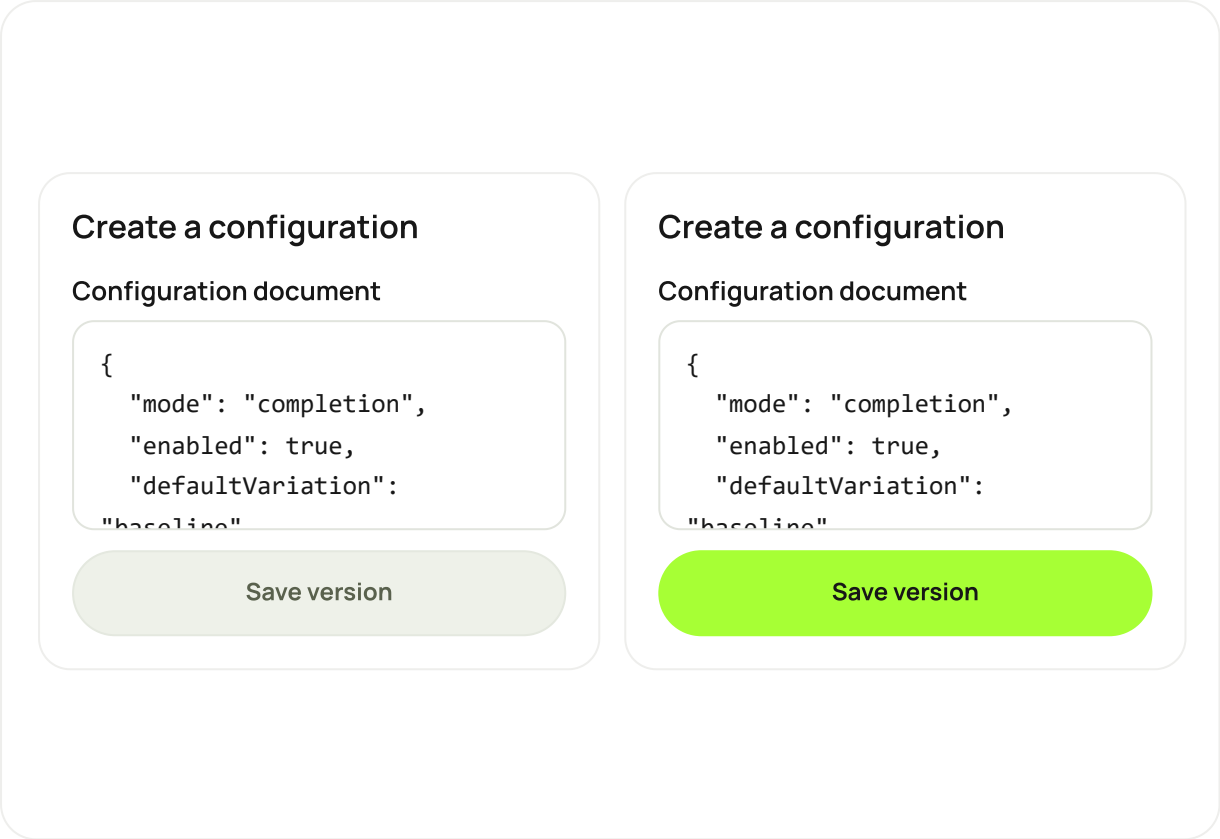
- **What you should see**
  Your safety policy is saved and can be inspected with its baseline and action.

1. Define increasing percentages ending at 100 with elapsed-time and candidate sample requirements. A running integration must report completed outcomes; no traffic means no advancement.
2. Configure quality, latency or cost thresholds with minimum samples and pause/rollback action. Optional statistical guards compare baseline/candidate with metric, alpha and tolerance; maxWaitSeconds handles missed sample deadlines.
3. Choose an existing rollback version, inspect alerts and audit after a safety action, and verify the resulting serving version. Missing measurements are never zero. Algorithms are independently implemented, not competitor proprietary equivalence.

Open this part of the app ↗

Adaptive triggers and one-step fallback retries

Location: Advanced configuration; application RuntimeClient.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

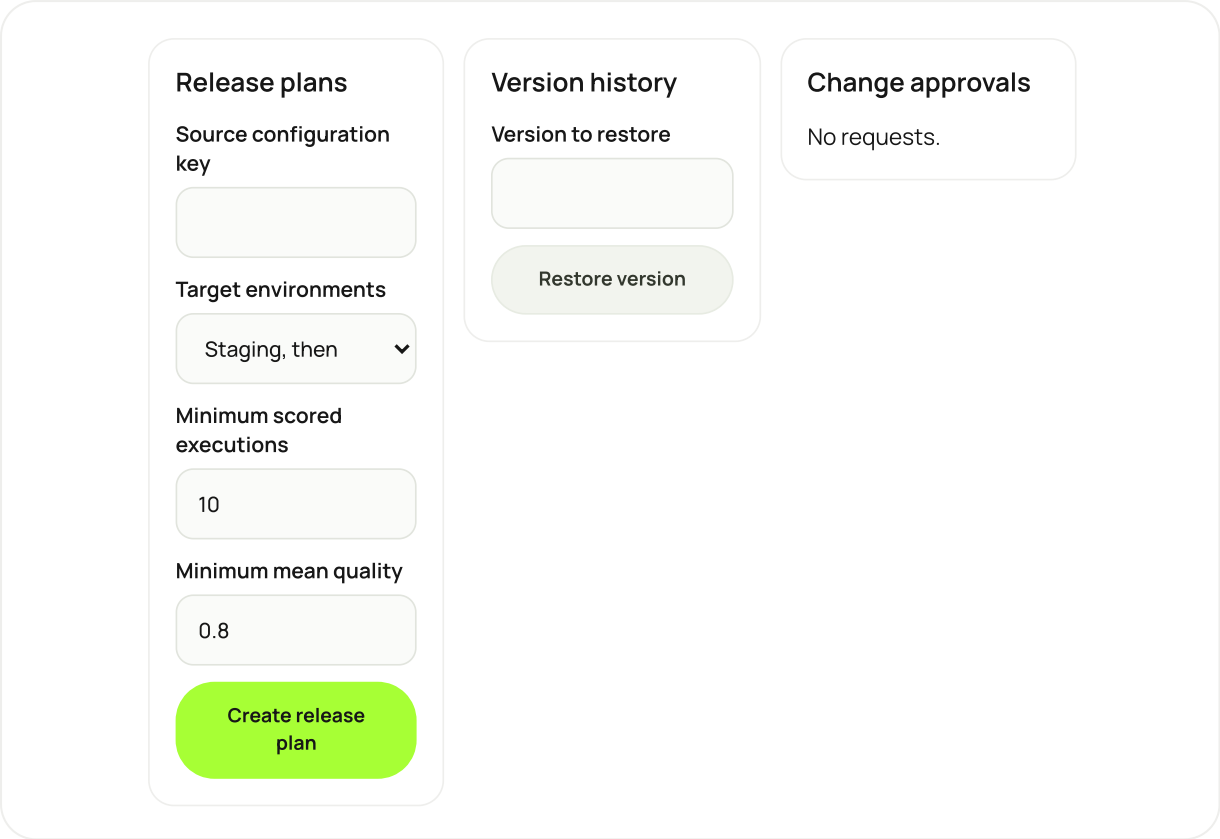
- **What you should see**
  Retry/fallback attempts retain the first outcome and their trace evidence.

1. Use measured-window adaptive triggers to switch future traffic after breach; configure delay, cooldown and repeat where appropriate. Inspect the new version and cleared competing policies after action.
2. Configure a fallback variation and minimum quality for a failed/low-quality first attempt. Execute, record the first outcome and request the linked retry.
3. Inspect parent trace ID and second-attempt evidence. Only one fallback is allowed; changed configurations block stale retries. A fully switched configuration does not endlessly trigger itself.

Open this part of the app ↗

History, stop, approvals, release plans and schedules

Location: AI runtime › Activity; Workspace › Releases.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

- **What you should see**
  A promoted, scheduled or reviewed release has an auditable status.

1. Restore from immutable history into a new version; configuration restore clears rollout/experiment automation. Stop serves the explicit safe default: it does not itself prohibit application model calls.

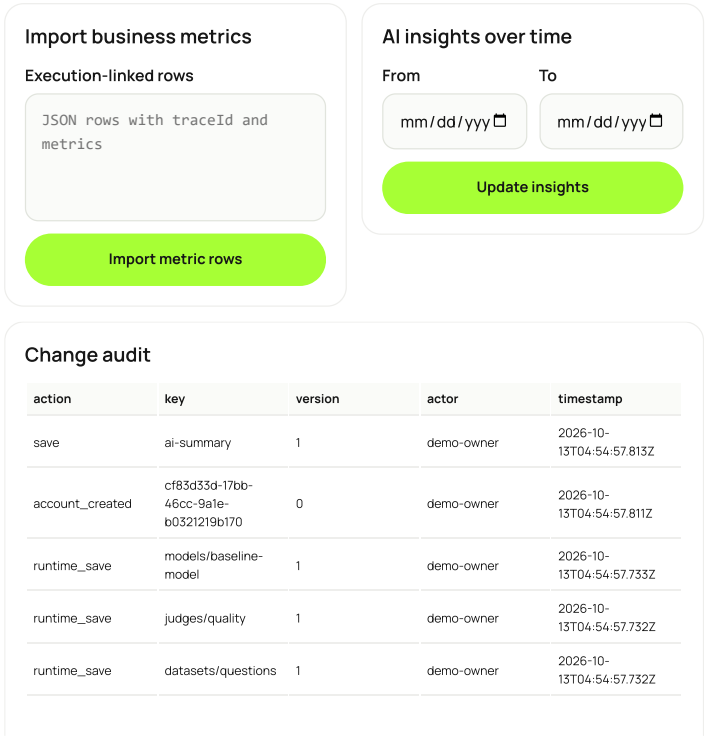
2. With production approval enabled, saves/restores and automatic winner changes require another eligible actor. Self-approval and stale overwrites are blocked. Emergency and preconfigured safety actions remain audited.

3. Create a release plan from a pinned source, choose stages and evidence gates, then advance after scored samples/quality pass. Dependencies are promoted with the config. Schedule timed changes with expected version; production schedules need independent review. Cancel pending plans/schedules when no longer needed.

Open this part of the app ↗

Monitoring, business metrics, audit and export

Location: AI runtime › Monitor/Activity; Workspace › Observability.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

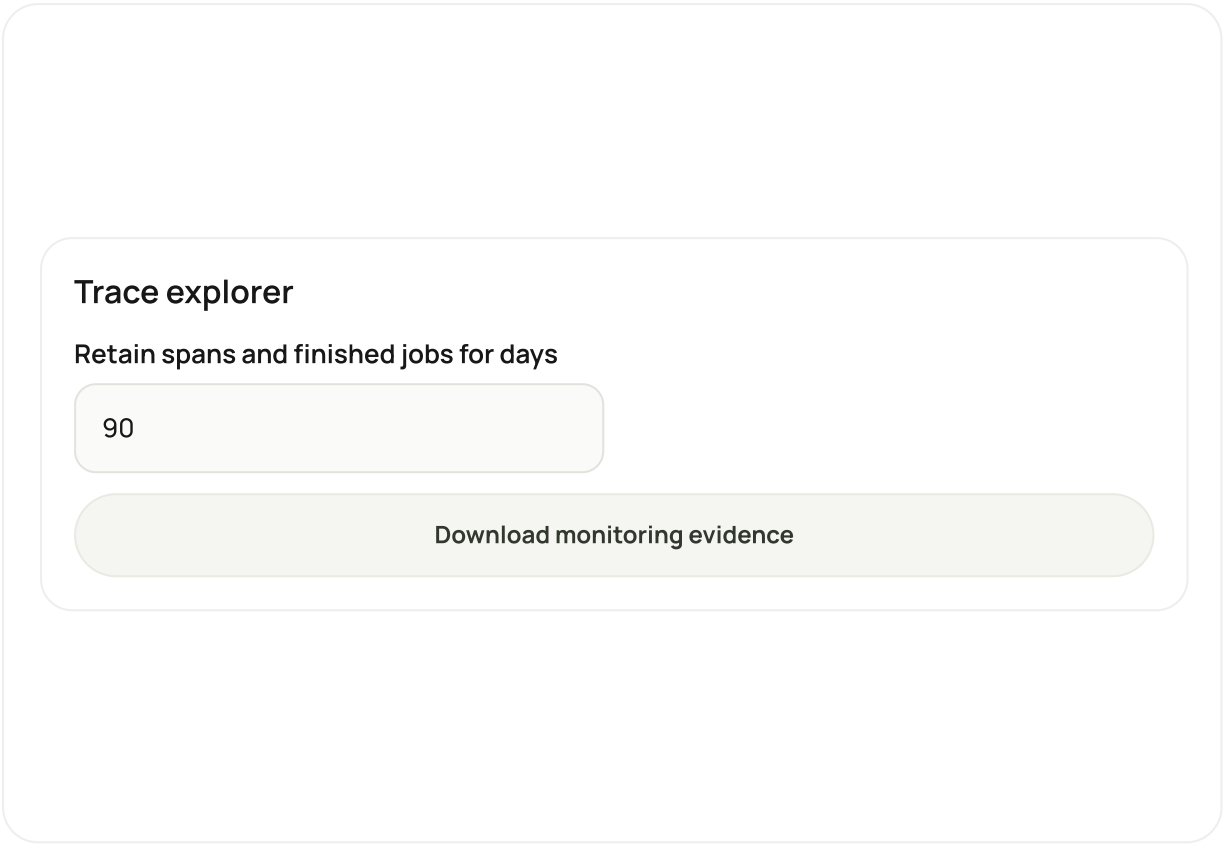
- **What you should see**
  Metrics, imports and exports expose actual recorded evidence.

1. Inspect current/historical version metrics, trends, variation evidence, charts, safety actions and graph metrics. Filter key/time window where available.
2. Import up to 500 execution-linked business-metric events. Review accepted, duplicate and rejected rows: imports can partially succeed and are not a direct warehouse-query connector.
3. Download JSON monitoring evidence containing insights, traces and audit. Retain incident evidence before cleanup; configuration and audit history survive the supplied retention cleanup.

Open this part of the app ↗

Traces and OpenTelemetry Collector

Location: Workspace › Observability › grouped trace explorer.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

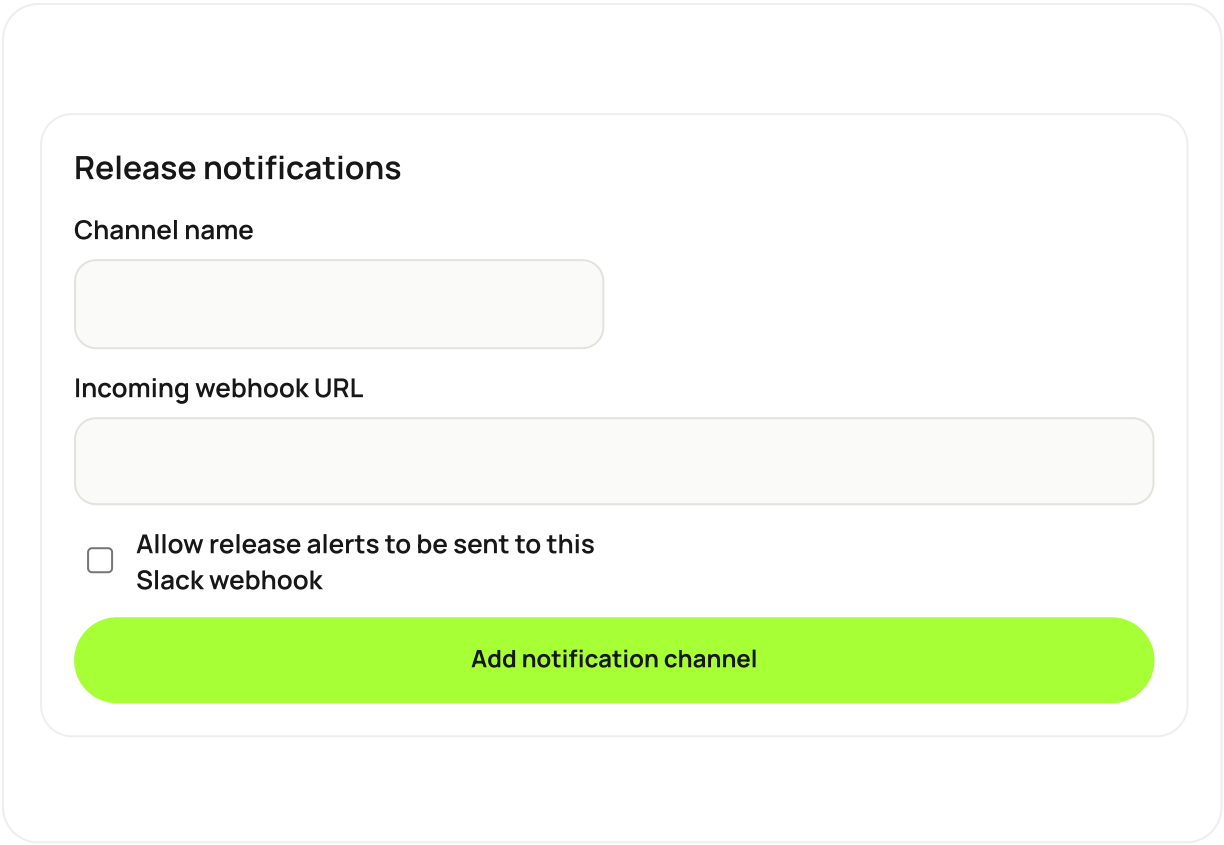
- **What you should see**
  Spans appear in the same workspace scope with parent/child relationships.

1. Submit OTLP JSON resource/scope spans or spans array to /api/runtime/traces. Use 32-hex trace IDs, 16-hex span IDs and Unix nanosecond timestamps; up to 200 spans per batch.
2. Link runtime.execution_id only to an execution in the same scope. Error status and supported attributes can produce outcomes; duplicates are idempotent and invalid links return warnings. Inspect service, parent/root, duration and raw evidence.
3. For OTLP protobuf/HTTP or gRPC use deploy/otel-collector.yaml with compose.telemetry.yaml. Issue scoped telemetry reader access and set endpoint/token. The Collector exports JSON to /api/runtime/otlp; the app itself is JSON-only.

Open this part of the app ↗

Notifications, Slack delivery and retention

Location: Workspace › Observability and Access & settings.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

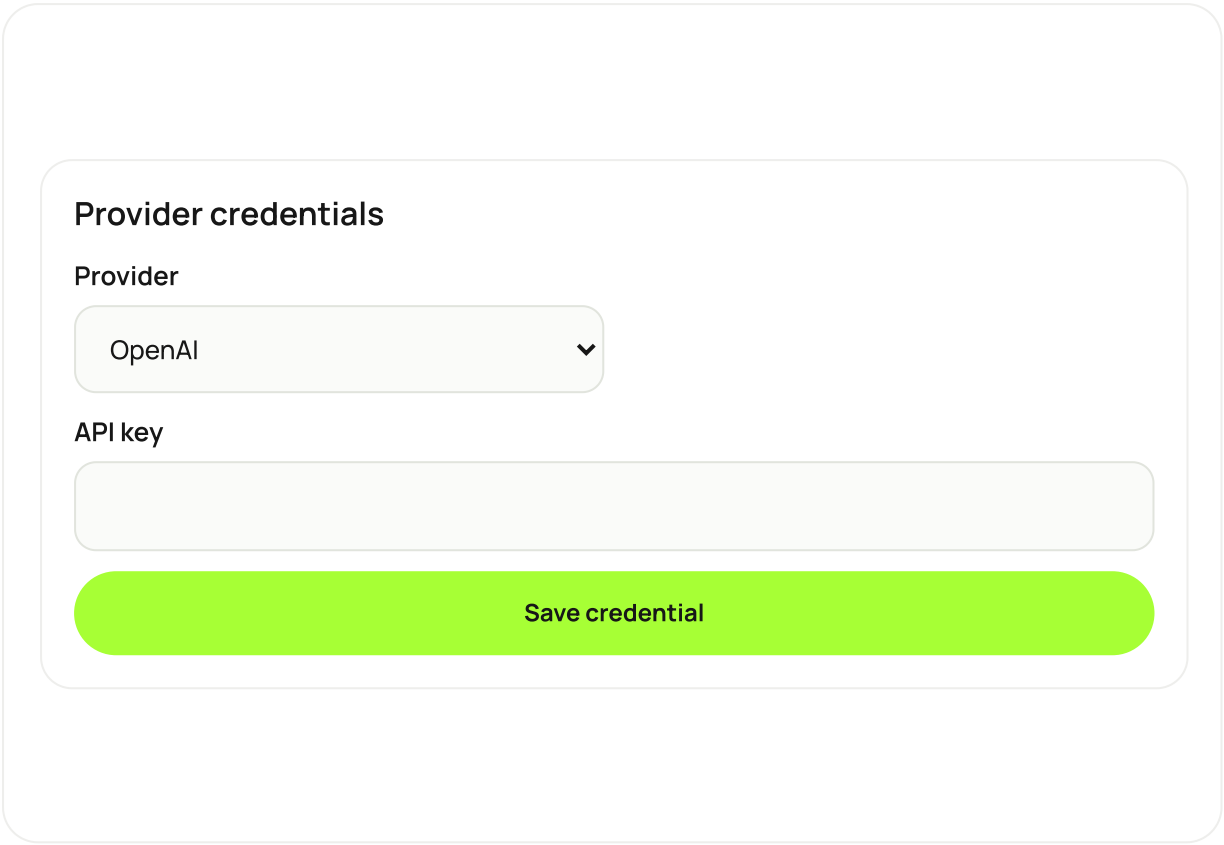
- **What you should see**
  Notification, delivery and retention evidence show their current state.

1. Inspect notification evidence and mark read after review. Administrator-configured Slack webhooks require explicit outbound opt-in; new channels skip old alerts.
2. Review outbox delivery state and retries. Backoff stops after five attempts. Real delivery needs your Slack organization; crashes can repeat external side effects.
3. Preview retention for old spans and finished jobs before explicit cleanup. Export or back up evidence needing longer retention; history and audit are retained.

Open this part of the app ↗

Provider credentials and live validation

Location: Workspace › Access & settings › Provider credentials.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

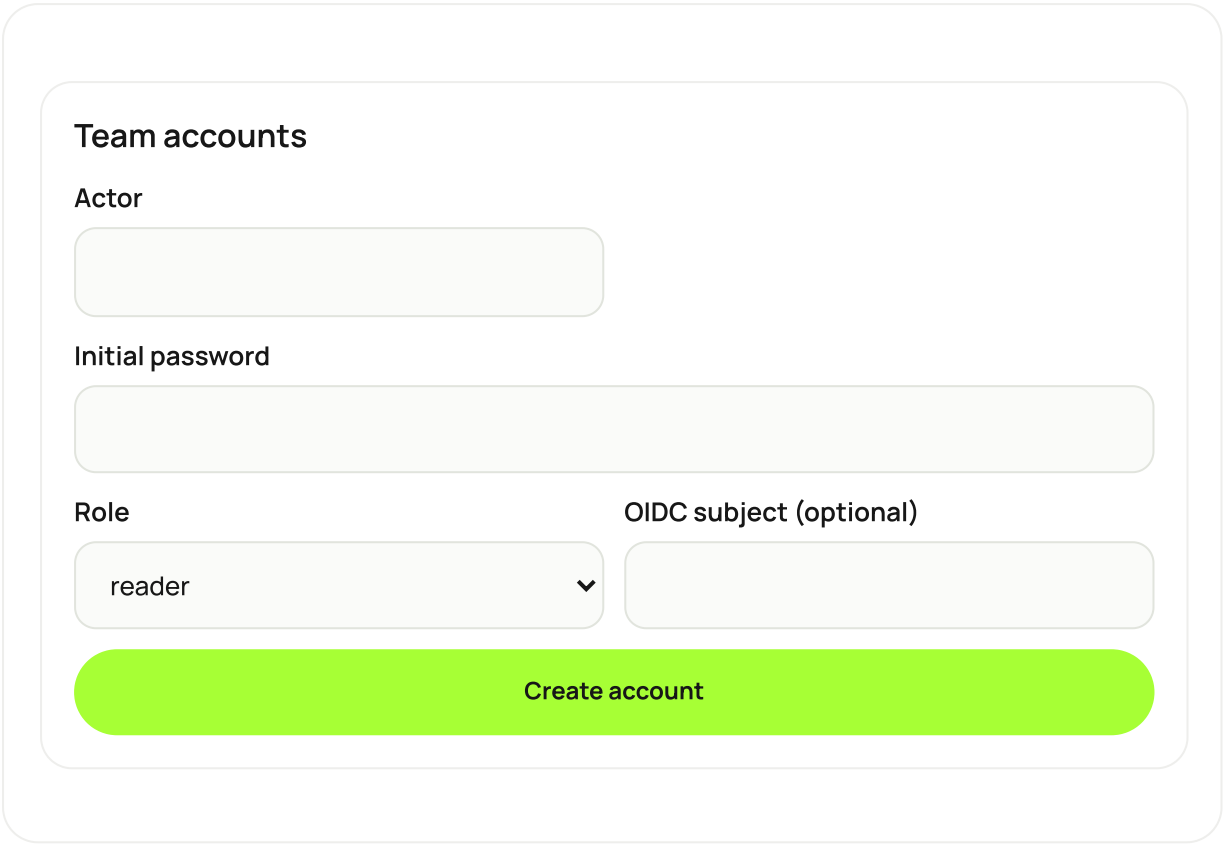
- **What you should see**
  The provider is configured without revealing its stored secret.

1. Save an OpenAI, Anthropic or Gemini key locally; values are encrypted and not returned. Or set provider environment variables and restart. Bedrock uses ENABLE_BEDROCK, AWS_REGION and the AWS credential chain.
2. Choose actual allowed model IDs/parameters. Removing a stored credential can fall back to an environment adapter. Validate pricing; unknown usage is not zero-cost evidence.
3. After configuring OpenAI, run scripts/live-openai-check.mjs with ALLOW_LIVE_PROVIDER_TEST=true and OPENAI_TEST_MODEL=gpt-4.1-mini for the authorized synthetic generation/judge smoke check. It can incur charges; no real credentials belong in recordings.

Open this part of the app ↗

API credentials, accounts and permissions

Location: Workspace › Access & settings.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

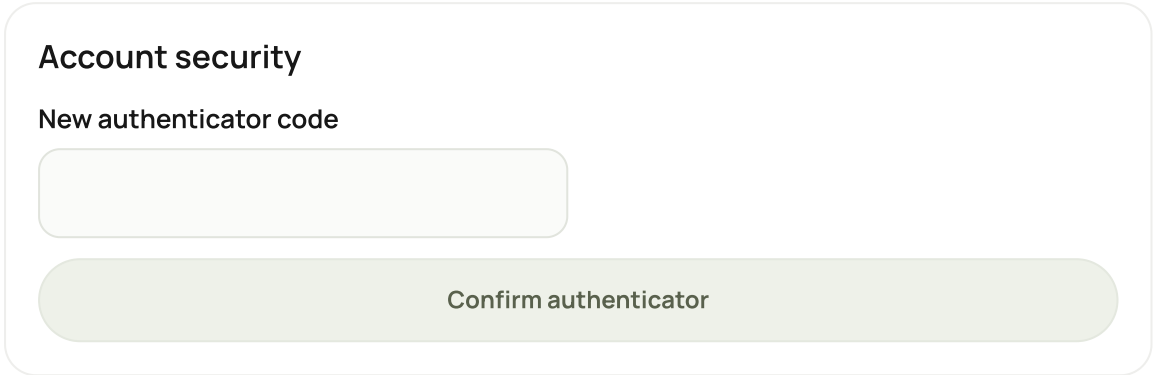
- **What you should see**
  The intended account or token has only its requested role and scope.

1. Issue expiring credentials with actor/role and environment/resource scopes; use * or narrow paths such as configs/support. Protect the one-time token and revoke when unused.
2. Create local team accounts with role/password and optional OIDC subject. Owner/admin manage access, operator cannot edit production, reviewer reviews and reader is read-only.
3. Use Account access to change another account role with expected-role conflict checks or link SSO. Changes revoke sessions; disabling also revokes access. An administrator cannot change their own role in this flow.

Open this part of the app ↗

MFA, recovery codes, password and SSO

Location: Workspace › Account security and configured /auth/sso.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

- **What you should see**
  Account security changes are confirmed and revoked sessions require a new sign-in.

1. Sign in with an account, supply current password/existing code, set up the authenticator and confirm a fresh six-digit code. Save one-use recovery codes when shown.
2. Change passwords with required proof; MFA activation and password changes revoke actor credentials. Replay protection applies to authenticator/recovery codes; API tokens are separate from interactive MFA.
3. Configure OIDC issuer/client/secret/redirect URI and provision exact subjects. SSO checks PKCE, nonce/state and signature. Set PUBLIC_ORIGIN to the exact HTTPS origin for deployed login and test your real identity provider.

Open this part of the app ↗

SCIM Users and Groups

Location: /scim/v2 APIs; Workspace › Enterprise provisioning.

Directory provisioning API

```
GET /scim/v2/ServiceProviderConfig
GET /scim/v2/ResourceTypes
GET /scim/v2/Users
GET /scim/v2/Groups
```

SCIM is an API workflow; account controls are managed separately.

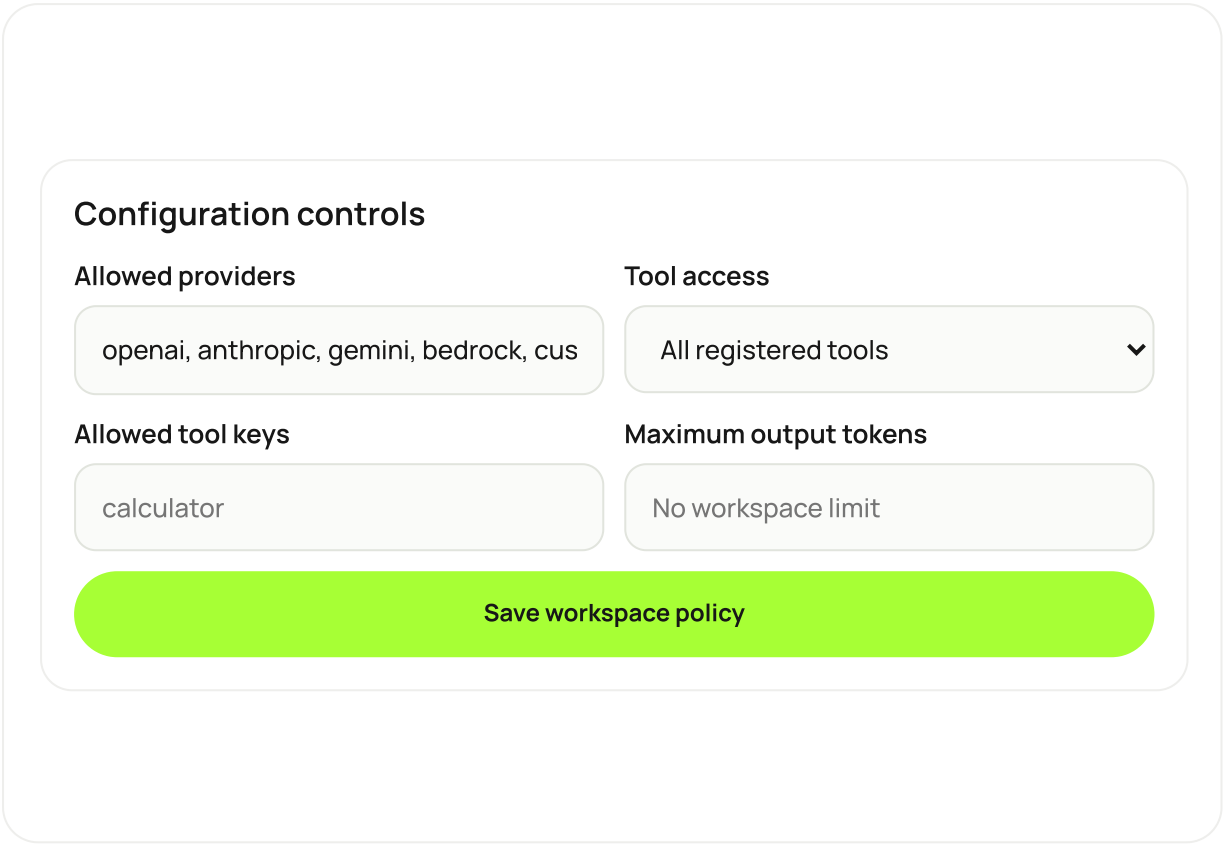
- **What you should see**
  Directory identities are provisioned without elevating roles through group membership.

1. Configure a full-workspace administrator credential in your directory client. Inspect ServiceProviderConfig, Schemas and ResourceTypes; provision Users as reader by default with no exposed password.
2. Create/read/list/replace/patch/delete Users and Groups. Use supported equality filters, pagination and If-Match ETags. Groups accept bounded tenant-local User members; add/remove or replace membership.
3. User deactivation/deletion revokes sessions. Group membership does not elevate application roles. Uniqueness, atomic patch failures and tenant boundaries are enforced; bulk, sorting and nested groups are not supported.

Open this part of the app ↗

Workspace execution policy

Location: Workspace › Access & settings › Workspace execution policy.



Selected controls from the actual product. Same labels, values, fonts and styling; arranged for print.

- **What you should see**
  The current environment policy applies to saved and existing configurations.

1. As owner/admin select environment, allowed providers and all registered tools or an explicit tool allowlist. Empty listed access denies all tools.
2. Optionally set an output-token cap and require an attached judge/guard. Variations must supply explicit positive output limits within the cap. Save with current policy version; audit records the change.
3. Retest existing configurations: save, resolve, retry and playground enforce current policy. Emergency disabled configs can be saved without bypassing execution checks. Separate judge token budgets are outside this cap.

Open this part of the app ↗

SDKs, frameworks, watch and signed cache

Location: src/runtime-sdk.js, src/frameworks.js, sdk/python, sdk/go, sdk/java.

Node runtime client

```
import { RuntimeClient } from './src/runtime-sdk.js';
import { providersFromEnvironment }
  from './src/providers.js';
const client = new RuntimeClient({
  baseUrl: 'http://127.0.0.1:4315',
  token: process.env.FLAG_ADMIN_TOKEN,
  environment: 'development',
  providers: providersFromEnvironment()
});
```

- **What you should see**
  Your application retrieves or executes configurations through its supported SDK.

1. Node supports execute, benchmarks, retries, graphs and optional tracing; Python supports HTTP/retry/graph/watch/signed fallback. Go and Java expose their supported client surfaces, not identical high-level APIs.

2. Use langChainExecutor with buildRunnable or langGraphExecutor with configurable.runtimeConfiguration. Integrating apps supply models, handler permissions and cancellation-aware execution.

3. Use watch for authenticated config-version SSE. Independently pin trustedPublicKey and tenant for signed snapshots. Cached fallback expires after 60 seconds, never masks authorization errors and uses safe defaults for automated/experimental configs; offline copies cannot receive immediate revocation.

Open this part of the app ↗

Self-hosted deployment, backup and recovery

Location: compose.production.yaml and scripts/recovery.mjs.

Production stack and recovery

```
docker compose -f compose.production.yaml up -d --build
# Open http://localhost:8080 after startup.

node scripts/recovery.mjs snapshot data NEW_SNAPSHOT
node scripts/recovery.mjs restore SNAPSHOT NEW_DATA
# Stop the server before switching its data directory.
```

1. Run npm ci/npm start with Node 24.14-24.x for development. For Docker set persistent STUDIO_ADMIN_TOKEN and run docker compose -f compose.production.yaml up -d –build; open localhost:8080.
2. For a real server configure domain/DNS/ports and exact STUDIO_PUBLIC_ORIGIN. Production-style Docker uses same-host workers, persistent volumes, Caddy, health checks, read-only app filesystem and bounded logs; it is not multi-region HA.
3. Snapshot database plus vault key using recovery.mjs snapshot DATA NEW_DIRECTORY. Protect off-host copies. Restore into a new directory with recovery.mjs restore SNAPSHOT NEW_DATA_DIRECTORY, stop the old server, verify data and switch volumes. Checksums detect corruption, not malicious replacement.

- **What you should see**
  The service is healthy, data persists, and a restore into a new directory validates.

Open this part of the app ↗

API access, troubleshooting and verification

Location: authenticated /api/runtime and current project sources.

Useful local checks

```
npm test
npm run check
node scripts/browser-check.mjs
node scripts/operations-browser-check.mjs
# Browser checks use isolated, synthetic fixtures.
```

- **What you should see**
  The request succeeds or returns an actionable authorization/version/provider error.

1. All writes require expectedVersion (0 for new). Resource APIs cover configs/datasets/judges/graphs/models/tools/segments/snippets, history and restore. Decision APIs include resolve/preview/retry/stop/metrics/watch, graph visits and linked outcomes.
2. Operations cover jobs/cancel/resume, experiments/sample analysis, release plans, calibrations, metric import, schedules/review/cancel, traces/otlp, notifications/read, channels, export and retention. Access covers secrets, credentials/revoke, accounts/disable/role/link-sso, MFA/password and policies.
3. For 401 reconnect; for 403 check role/scope/policy; for conflict refresh; for provider errors inspect actual model/credential/parameters; for stalled jobs check worker. Run npm test, npm run check and supplied browser/SDK/container checks. Local passing tests are not a hosted availability guarantee.

Open this part of the app ↗

Record a safe walkthrough

Location: your signed-in Chrome/Edge Tango extension.

Recording checklist

```
Use synthetic data only.
Exclude keys, tokens, MFA codes and recovery codes.
Record one workflow at a time.
Review every step before sharing.
Supplement API features with developer instructions.
```

1. Open the project in the browser profile where Tango is installed, start capture, follow one focused workflow, then finish and review the captured steps.
2. Only record synthetic safe data. Exclude credentials, MFA setup and recovery codes. Supplement API/SDK-only capabilities with developer instructions because they have no equivalent UI-only recording.
3. Use your available Tango PDF export for captured workflows. This handbook is locally authored from inspected code and verified UI evidence; it is not labelled a Tango export unless actual Tango capture/export succeeds.

- **What you should see**
  The recording has safe data and describes only the workflow actually performed.

When something does not work.

Site cannot be reached

The local server is stopped. From the project folder run npm start. Reuse your existing FLAG_ADMIN_TOKEN when restarting; then open http://127.0.0.1:4315/.

No completion generated

Resolve is a selection preview. To generate text, save a provider key, choose a valid provider/model, save the configuration, and explicitly allow provider calls in Test.

401 or disconnected

Reconnect with a valid workspace token or sign in again. Check credential expiry and whether the server token changed after a restart.

403 or a disabled action

Check role, environment, resource scope and execution policy. An operator cannot edit production; production changes can require an independent reviewer.

Version conflict or stalled job

Refresh the current resource before saving. For a queued job, inspect worker state and provider errors before retrying or resuming.

Provider error or unknown cost

Check the returned error, account quota, API key, model ID and parameters. Unknown token usage or missing configured prices means unknown cost, not free usage.

Keep this beside your first release.

Before you go live.

1. Use a scoped application token and the intended environment. Keep keys and tokens out of client-side code, screenshots and recordings.
2. Save a known safe baseline, resolve representative users, and evaluate a dataset before a rollout. Check your fallback/default variation.
3. Review permissions and production approval, inspect monitoring after release, and keep an independently protected recovery snapshot.

What this guide covers

The implemented feature areas in the index. Control examples are native HTML/CSS shapes from an isolated synthetic workspace, printed as vector graphics with searchable text. They retain the project's styling and labels.

What still needs your setup

Real provider accounts, external OIDC/SCIM/Slack services, production hosting and operational evidence. The local product does not establish a hosted SLA, certification or full enterprise competitor equivalence.

About this guide

Locally authored from current product screens and project source documentation. It is not a Tango export. No real tokens or provider keys are included.

AI runtime ↗ Evaluations ↗ Workspace access ↗